Discovery Executive Summary

Project: php-admin-discovery · **Generated:** 31/07/2026, 19:02:16

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk F5 (Legacy/Procedural patterns across all 4 PHP files) and H10 (5+ hardcoded business/runtime values in view templates), with Moderate H1 and F1 amplifying the single-file bottleneck in header.php.

EXECUTIVE SUMMARY

The `php-admin-panel` repository is a minimal 4-file PHP admin dashboard template built with plain procedural PHP — no framework, no ORM, no database, and no layered architecture of any kind. The most severe hotspot is **F5 (Legacy Component Patterns)**: every PHP file uses procedural includes with hardcoded data and no OOP, namespaces, or autoloading, meaning any extension of this template will immediately inherit all of these structural deficits. `header.php` is the second major risk area: it acts simultaneously as menu configuration, page-routing logic, and full HTML renderer (~170 LOC), violating the Single Responsibility Principle and making future decomposition significantly harder. Because this is an intentional starter template rather than a production system, hotspots like Missing Repository Pattern and Shared Database Coupling are not yet present — but the absence of any structural guidance means contributors are likely to add new features directly into existing PHP files, compounding technical debt rapidly.

§1.1 Benchmark Ratings Summary

Layers covered: Backend — 4 PHP files (procedural includes, no framework, ~280 total LOC); Frontend — PHP server-side templates with embedded JavaScript (CDN-loaded SweetAlert2 in `footer.php`). No React/Vue/Angular SPA detected. No database layer present.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~170 LOC (`header.php` as de-facto handler)	MODERATE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	0 direct accesses (no repos/models exist — service layer entirely absent)	MODERATE
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	0 — no database exists in this template	GOOD
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	0	GOOD
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (no SQL anywhere in codebase)	GOOD
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 — largest file is `header.php` at ~170 LOC	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	0 — single domain, no boundaries defined	GOOD
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	0 — no database	GOOD
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	~170 LOC (`header.php` mixes routing logic + HTML rendering)	MODERATE
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	0 — no fetch/axios/XHR calls anywhere	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 — `header.php` is	GOOD

						~170 LOC, under threshold	
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2 levels	3–4 levels	>4 levels	1 level (PHP include scope, variables set and consumed in `header.php` only)	GOOD
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	4 of 4 PHP files — procedural includes, no OOP, hardcoded values, no autoloader	HIGH RISK
H10	Hardcoded Data / No Config Layer (additional)	Config values hard-coded in view files (target: 0)	0	1–3	>3	5+ hardcoded values: username, 4 stat card numbers, logout URL `/logout/`	HIGH RISK

---

§1.4 Actions Required

Hotspot	Action	Rating	Priority
F5 — Legacy / Procedural PHP Patterns	Add `composer.json` with PSR-4 autoloading (`"App\\": "src/"`); introduce `src/Controllers/` , `src/Services/` , `config/` directory structure; convert flat page scripts to Controller + Template pattern	HIGH RISK	CRITICAL
H10 — Hardcoded Data / No Config Layer	Create `config/app.php` with `APP_LOGOUT_URL` constant; move stat card values to `DashboardService::getStats()` ; replace hardcoded username with `$_SESSION['user']['name'] ?? 'Guest'` ; fix broken `/logout/` route	HIGH RISK	CRITICAL
H1 — Fat De-facto Handler	Extract `$menuItems` to `config/navigation.php` ; extract breadcrumb + active-state logic to `src/Services/NavigationService.php` ; keep `header.php` as pure HTML layout template	MODERATE	HIGH
F1 — Business Logic in PHP Templates	Move page-context resolution out of `header.php` into `NavigationService::resolve()` ; pre-compute `is_active` boolean per menu item so the template performs zero logic	MODERATE	HIGH
H2 — Missing Service Layer	Once Composer + Controllers are in place (Phase 3), add `src/Services/DashboardService.php` and `src/Services/AuthService.php` so future database queries and user logic have a designated home	MODERATE	MEDIUM

---

§1.5 Expected Outcomes

- **Testability:** With `NavigationService` extracted as a pure class, breadcrumb and active-menu logic can be unit-tested with PHPUnit in milliseconds — no web server, no HTML rendering required.
- **Safe extensibility:** New pages register a route and a controller method rather than editing `header.php`, eliminating the single-file bottleneck and merge conflicts as the project grows beyond 2 pages.
- **Configuration-driven:** Moving stat cards and user identity to a service/session layer lets any consumer of this starter template update runtime values in `config/` — not by hunting through view files.
- **Framework-ready:** Introducing Composer and PSR-4 in Phase 1 reduces a future migration to Laravel, Slim, or Symfony from a full rewrite to a progressive refactor — controllers and services can be ported one at a time.
- **No broken production paths:** Fixing the hardcoded `/logout/` route (which currently 404s) and the placeholder stat-card numbers (150, 53%, 44, 65) prevents those values from being shipped to real users by developers who missed them during customization.

Report saved to `docs/discovery/01-architecture-design.md`. The orchestration UI will convert it to PDF automatically. Run artifact written to `agent-runs/20260731T185413_lr8ly6/01-architecture-design-artifact.md`.